

LEARN PYTHON PROGRAMMING – BEGINNER TO MASTER

Section: 4. Operators and Expressions

Lecture: 27. Arithmetic Operators

Section 1: Lecture Summary

This lecture focuses on **Python operators**, specifically the arithmetic operators. It begins with an overview of the various operator types available in Python, highlighting arithmetic, assignment, relational, logical, bitwise, and more. The main content delves into arithmetic operators, explaining their usage, syntax, and behavior in Python, emphasizing differences such as floor division and modulus operators, which are distinctive in Python compared to some other languages. The lecture covers addition, subtraction, multiplication, division (float and floor), modulus (remainder), and exponentiation (power). The notions of operands, operators, and expressions are introduced for clarity. Practical examples are demonstrated, including code snippets executed in Jupyter Notebook to solidify understanding.

Section 2: Key Concepts and Explanations

- Operators and Operands

- An **operator** is a symbol that performs a computation or action, such as '+' for addition.
- An **operand** is the data or variable upon which the operator acts (e.g., 'a' and 'b' in 'a + b').
- An **expression** combines operands and operators to compute a value, e.g., 'c = a + b'.

- Arithmetic Operators in Python

- **Addition (+)**: Adds two operands. Example: 'c = a + b' adds 'a' and 'b'.
- **Subtraction (-)**: Subtracts second operand from the first. 'c = a - b'. Flips sign if order is reversed.

- **Multiplication (`*`)**: Multiplies operands. Works like usual mathematical multiplication.

- **Division (`/`) [Float Division]**: Divides and returns a floating-point number. For example, `14 / 4` results in `3.5`.

- **Floor Division (`//`)**: Divides and returns the quotient truncated down to the nearest integer (ignores remainder). For example, `14 // 4` results in `3`. Also called integer division.

- **Modulus (`%`)**: Returns the remainder of division. For `14 % 4` the result is `2`.

- **Exponentiation (`**`)**: Raises a number to the power of another, e.g., `2 ** 5` represents $(2^5 = 32)$.

- Additional Details

- Floor division and modulus together handle quotient and remainder separately.

- Python differentiates between `/` and `//` operators unlike many other languages where only one division operator exists.

- Negative values in multiplication or subtraction behave according to standard arithmetic rules (signs multiply or subtract accordingly).

- Using float values in division results in float outputs regardless of operator.

Section 3: Example Code and Use Cases

Addition

```
a = 10
b = 15
c = a + b
print(c) # Output: 25
```

Adds 10 and 15, stores result in c.

Subtraction

```
a = 20
b = 15
c = a - b
print(c) # Output: 5
```

```
c = b - a
print(c) # Output: -5 (negative result due to order)
```

Demonstrates subtraction and effect of reversing order.

Multiplication

```
a = 4
b = 7
c = a * b
print(c) # Output: 28
```

Multiplies 4 and 7.

Floor Division and Modulus

```
a = 14
b = 4
quotient = a // b
remainder = a % b
print(quotient) # Output: 3
print(remainder) # Output: 2
```

Floor division gives integer quotient; modulus gives remainder.

Float Division

```
a = 14
b = 4
result = a / b
print(result) # Output: 3.5
```

Standard division producing floating-point result.

Exponentiation (Power)

```
base = 2
exponent = 5
power = base ** exponent
print(power) # Output: 32
```

Calculates 2 to the power of 5.

Float value division example

```
a = 14.8
b = 4
print(a / b) # Output: 3.7
```

Division involving float value resulting in float output.

Section 4: Key Takeaways

- Python supports multiple operator types; this lecture focuses on **arithmetic operators**.
- Operators work on operands to form expressions that compute values.
- Arithmetic operators include `+`, `-`, `*`, `/`, `//`, `%`, `**`.
- `/` is **float division**, always returns a float (even with integers).
- `//` is **floor division**, returns integer quotient by truncation (floor).
- `%` provides the remainder from division.
- `**` performs exponentiation (power).
- Python uniquely distinguishes float division (`/`) and floor division (`//`).
- Expressions can be tested interactively in environments like Jupyter Notebook to validate results.
- Understanding the behavior of these operators with negative and float values is important for precise calculations.